



## SOURCES & MONITORS

# How rays are born, and how they're recorded

A closer look at source and monitor components in McStas & McXtrace

McStas — neutrons

McXtrace — X-rays



Peter Willendrup · DTU Physics

PaNRAID School · supported by PEPR DIADEM



# What we'll cover

Building on the Common Introduction — now with a closer look at two component families every instrument needs.

1

## What a source does

Setting the ray state at the start of the beamline

3

## Anatomy of a simple source

Position, direction, and spectrum

5

## Shared source physics

Solid angle, ncount, and statistical weight

7

## Monitors vs. detectors

Probing the beam without disturbing it

2

## Finding the docs

mcdoc & mxdoc, and how to read a component's parameters

4

## Two source landscapes

Mathematical, reactor/pulsed, and accelerator-based sources

6

## MCPL

Coupling simulations together via particle lists

8

## A shared monitor library

1D, 2D and nD monitors — and how they compute statistics



# What a source component does

- A source generates rays — in McStas terms, it sets the ray state to something representative of the source being modelled.
- In practice: draw values for the ray's state vector from chosen distributions — not from a single fixed value.
- Example: rays of uniformly distributed wavelength, emerging from a circular aperture, aimed at a region of interest.
- Every other component downstream only ever sees a ray in this state — it doesn't know or care how the source produced it.

## Recap — the ray state vector

### McStas · neutron ray

$$\{ x, y, z, \quad vx, vy, vz, \quad t, \quad sx, sy, sz, \quad p \}$$

*position · velocity · time · spin · weight*

### McXtrace · photon ray

$$\{ x, y, z, \quad kx, ky, kz, \quad Ex, Ey, Ez, \quad \varphi, \quad t, \quad p \}$$

*position · wave vector · polarisation · phase · time · weight*



# Finding the docs

Every component's parameters, defaults and physics are documented — and the two codes use the same pattern to reach them.

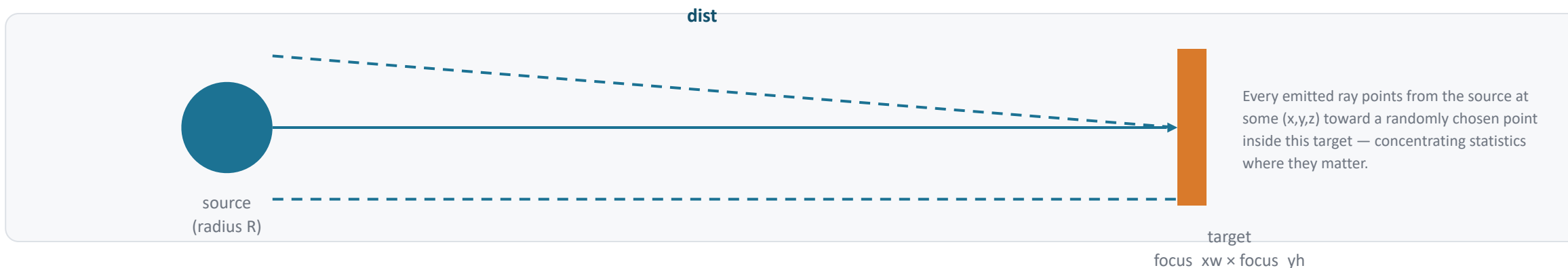
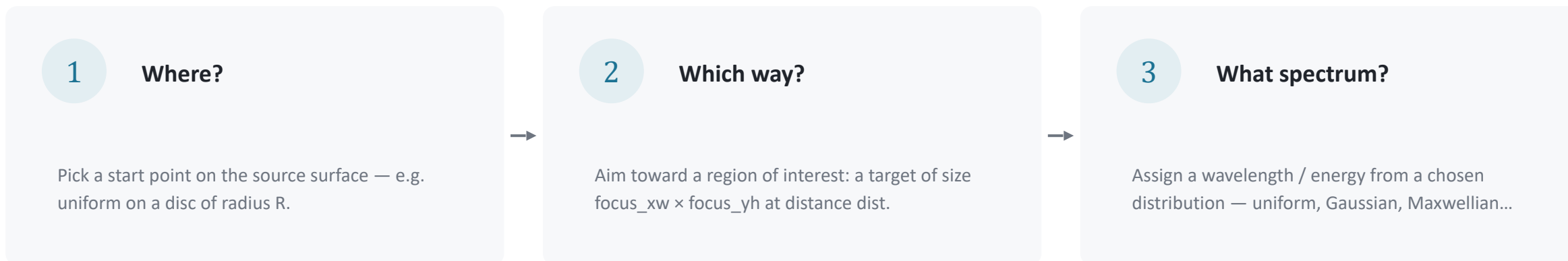
| You want...                     | McStas                                    | McXtrace                                  |
|---------------------------------|-------------------------------------------|-------------------------------------------|
| Overview documentation page     | <code>mcdoc</code> (or "Docs" in mcgui)   | <code>mxdoc</code> (or "Docs" in mxgui)   |
| The full PDF component manual   | <code>mcdoc -c</code>                     | <code>mxdoc -c</code>                     |
| Docs for one specific component | <code>mcdoc &lt;component_name&gt;</code> | <code>mxdoc &lt;component_name&gt;</code> |

**Watch out:** online copies of the manuals exist ([mcstas.org](http://mcstas.org) / [mcxtrace.org](http://mcxtrace.org)) but may not match your installed version exactly — the local `mcdoc` / `mxdoc` is always authoritative.



# Anatomy of a simple source

Three questions every source answers — identically in shape, whichever code you're in.





# A minimal source, in both codes

```
COMPONENT src = Source_simple(  
    radius=0.05, lambda0=2.5, dlambda=1.5,  
    focus_xw=0.1, focus_yh=0.1, dist=5)  
AT (0,0,0) RELATIVE origin
```

McStas · Source\_simple

```
COMPONENT src = Source_flat(  
    radius=0.05, lambda0=2.5, dlambda=1.5,  
    focus_xw=0.1, focus_yh=0.1, dist=5)  
AT (0,0,0) RELATIVE origin
```

McXtrace · Source\_flat

**Same parameter names, same units:** radius, lambda0/dlambda, focus\_xw, focus\_yh and dist mean the same thing in both instrument files — only the wavelength range and physical scale differ between neutrons and X-rays.



# Two source landscapes

This is where the two libraries look most different — shaped by very different facilities.

## McStas · neutron sources

### MATHEMATICAL

`Source_simple`, `Source_div`

### REACTOR / CONTINUOUS

`Source_Maxwell_3`, `Source_gen`

### PULSED (SPALLATION)

`ESS_butterfly`, `Moderator`, `SNS_source`, `ISIS_moderator`

### PARTICLE-LIST I/O

`MCPL_input` / `MCPL_output`

## McXtrace · X-ray sources

### MATHEMATICAL

`Source_flat`, `Source_div`, `Source_pt`

### ACCELERATOR INSERTION DEVICES

`Bending_magnet`, `Wiggler`, `Undulator`

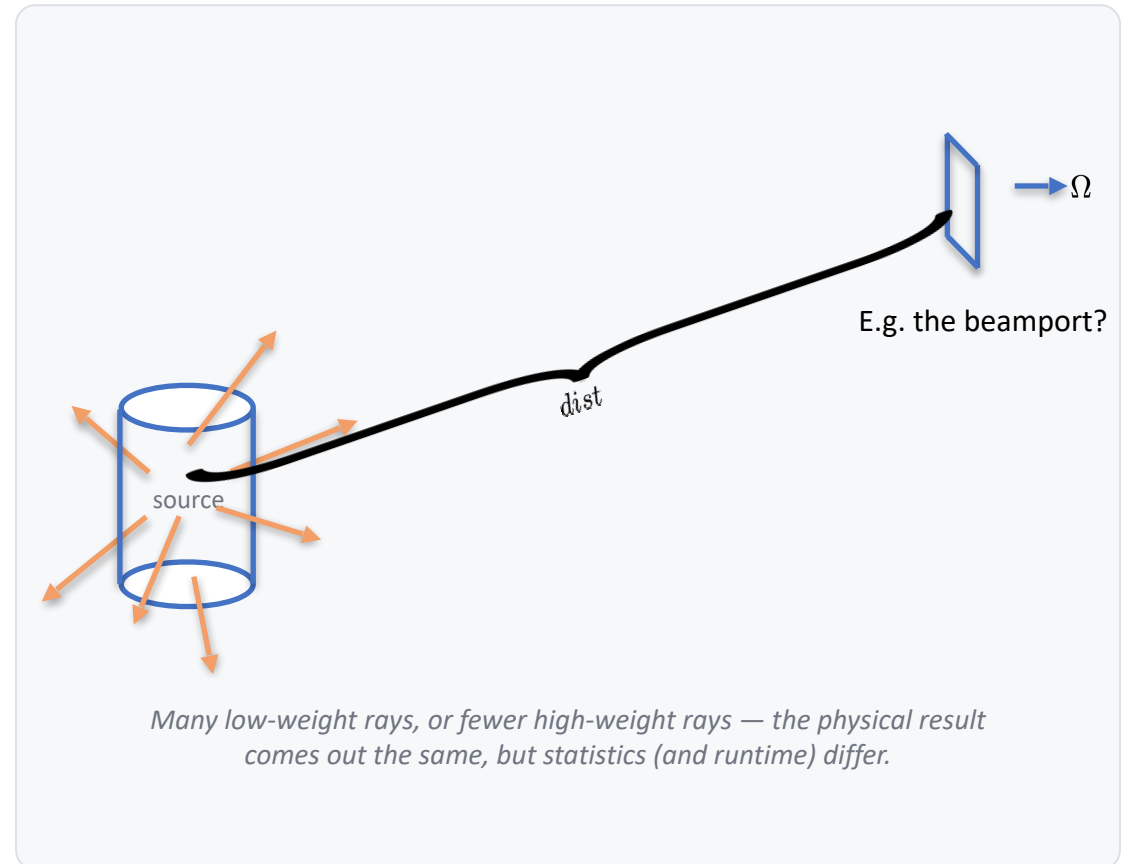
### PARTICLE-LIST I/O

`MCPL_input` / `MCPL_output`

*Neutron sources mirror facility diversity (reactor vs. spallation); X-ray sources mirror accelerator physics (magnet type). Both share simple analytic sources and the same MCPL format.*

# Solid angle, ncount, and ray weight

- Most neutron- and some X-ray sources emit into the full  $4\pi$  steradian — but a simulation only needs the solid angle  $\Omega$  subtended by the beamline's first aperture.
- The emission integrated over  $\Omega$ , wavelength and time gives a physical intensity (rays / second).
- That intensity is partitioned over a finite number of simulated rays: ncount (default 1e6, set with -n / --ncount).
- Each ray carries a statistical weight  $p$ , chosen so the sum of weights over all rays reproduces the physical intensity.





# MCPL: coupling simulations together

**MCPL\_output** writes the full ray state to a portable, code-agnostic file at any point in an instrument.

**MCPL\_input** reads those rays back in as a source — in the same code, or a different one entirely (McStas ↔ McXtrace ↔ MCNP ↔ Geant4...).

```
COMPONENT vout = MCPL_output(filename="v.mcpl", doubleprec=1, polarisationuse=1)
AT (Xout,Yout,Zout) RELATIVE PREVIOUS

COMPONENT vin = MCPL_input(filename="v.mcpl", polarisationuse=1)
AT (Xin,Yin,Zin) RELATIVE PREVIOUS
```

**Coordinates travel with the rays:** MCPL stores an implicit translation/rotation, so a ray written out at one orientation is placed correctly when read into an instrument with a different local frame.



# Pulsed neutron sources

Reactors run continuously; spallation sources are pulsed. McStas offers a spectrum of ways to model that.

## The simplest trick

Use a continuous source and force an infinitely short pulse:  $t = 0$  or define a short pulse via a `rand01()` for every ray within an **EXTEND** block — or use a chopper to carve pulses out of a continuous beam.

```
COMPONENT src = Source_simple(...)
AT (0,0,0) RELATIVE origin
EXTEND %{
    t = 10e-3*rand01(); // 1 ms square pulse
%}
```

## Dedicated pulsed-source components

**Moderator** flat pulsed source, uniform energy spectrum, energy-dependent pulse shape

**ESS\_butterfly** ESS's characteristic “butterfly” moderator brightness model

**SNS\_source / SNS\_source\_analytic** samples MCNP target+moderator tallies (or fast Padé fits); also used for J-PARC, CSNS

**ViewModISIS / ISIS\_moderator** samples tallies supplied per beamport at ISIS



# Monitors vs. detectors, in general

Both codes draw exactly the same line — and let you program either behaviour when you need to.

## Monitors

- An intensity probe of the beam
- Should be transparent to rays — efficiency  $\ll 1\%$  (neutrons) / low efficiency (X-rays)
- Doesn't meaningfully perturb what happens downstream

## Detectors

- Should detect (almost) every ray — efficiency as high as possible
- McStas models a real He3 tube via PSD\_Detector
- Otherwise: both codes let a monitor be given any efficiency/transparency you specify

**Same philosophy in both codes:** “we refer to both indistinguishably as monitors” — the component is the same, only the parameters you give it change its role.

**Simulation of 'real' detection-physics is an exception:** “we leave that to expert-codes such as Geant4” — measurement is by default “magic / for free”



# Adding monitors to a beamline

Swap the source component and the rest reads identically in McStas or McXtrace.

```
COMPONENT origin = Progress_bar()    AT (0,0,0) ABSOLUTE

COMPONENT src = Source_simple(radius=0.05, lambda0=2.5, dlambda=1.5,
                             focus_xw=0.1, focus_yh=0.1, dist=5)

AT (0,0,0) RELATIVE origin

COMPONENT psd = PSD_monitor(xwidth=0.2, yheight=0.2, filename="psd.dat")
AT (0,0,5) RELATIVE src

COMPONENT lm = L_monitor(xwidth=0.2, yheight=0.2, filename="lm.dat", Lmin=0, Lmax=8)
AT (0,0,5.01) RELATIVE src
```

**In McXtrace:** replace `Source_simple` with `Source_flat` — `PSD_monitor` and `L_monitor` are called exactly the same way.



# A shared monitor library

Most monitors exist under the same name and parameter conventions in both codes.

## 1D

L\_monitor  
TOF\_monitor  
Hdiv\_monitor  
E\_monitor  
MeanPolLambda \*

## 2D

PSD\_monitor  
PSD\_monitor\_4PI  
Divergence\_monitor  
DivPos\_monitor  
PolLambda\_monitor \*

## nD

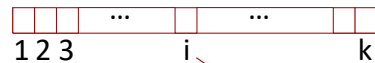
Monitor\_nD  
— records almost  
any requested  
quantity, any  
shape

*\* a small number of monitors are probe-specific — polarisation monitors exist for neutrons, not X-rays. Look up the full, current list with mcdoc monitor / mxdoc monitor.*



# How a monitor computes its statistics

- Every bin (pixel) accumulates a running sum of ray weight as the simulation proceeds — not a per-ray record.
- At the end of the run, each bin reports three numbers: intensity  $I$ , its estimated RMS error  $E$ , and an effective event count  $N$ .
- $I$  is the weighted sum of rays landing in that bin;  $E$  follows from the sum of squared weights — the Monte Carlo estimate of statistical uncertainty.
- The same estimator runs underneath every monitor, in both codes — from a single `L_monitor` to the full `Monitor_nD`.



$$I = \sum_N p_j$$

In bin  $i$ ,  $N$  events each carrying a fractional intensity  $p_j$  so that

$I(\lambda)$ : intensity per bin

$I = \sum p_j$  (summed weight)

$E \approx \sqrt{\sum p_j^2}$  (RMS error)

$N$  = ray count in bin  $i$





# Monitor\_nD: the swiss-army-knife

A general 0D / 1D / 2D recorder — almost any shape, recording almost any requested quantity — same options grammar in both codes.

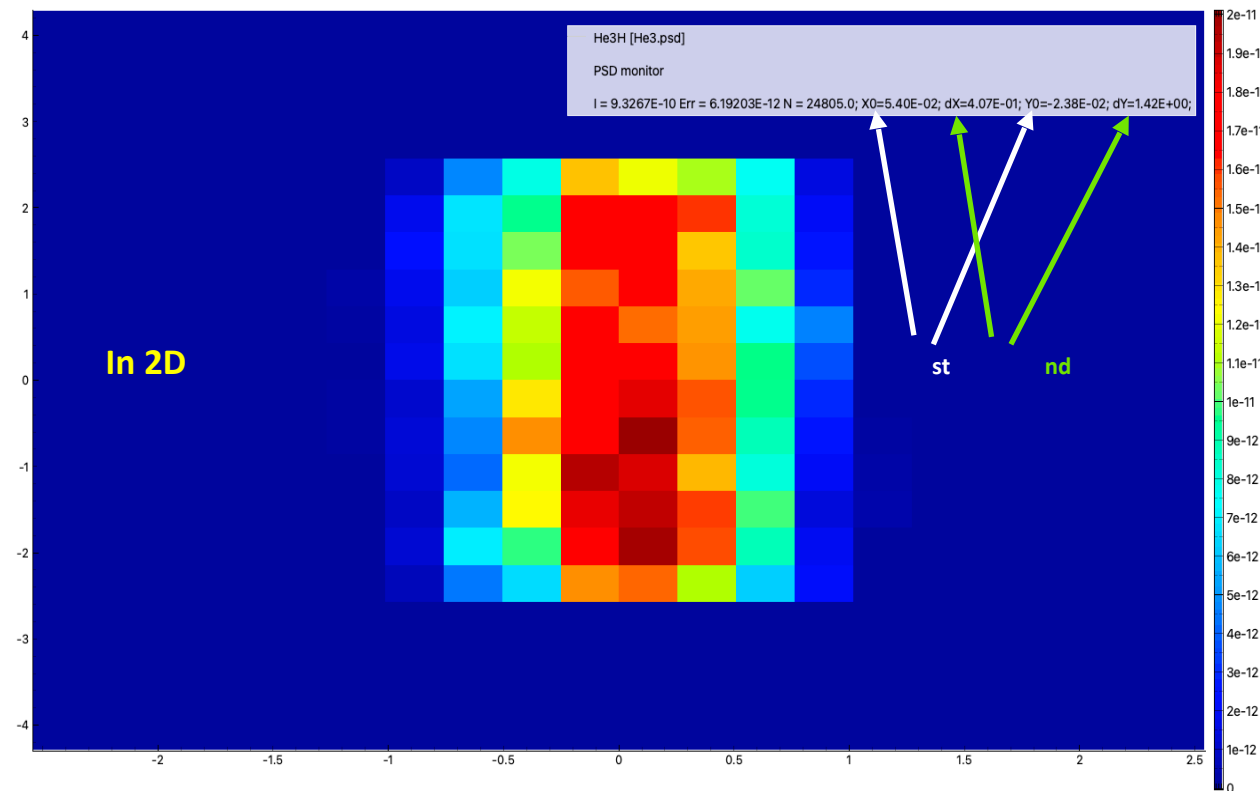
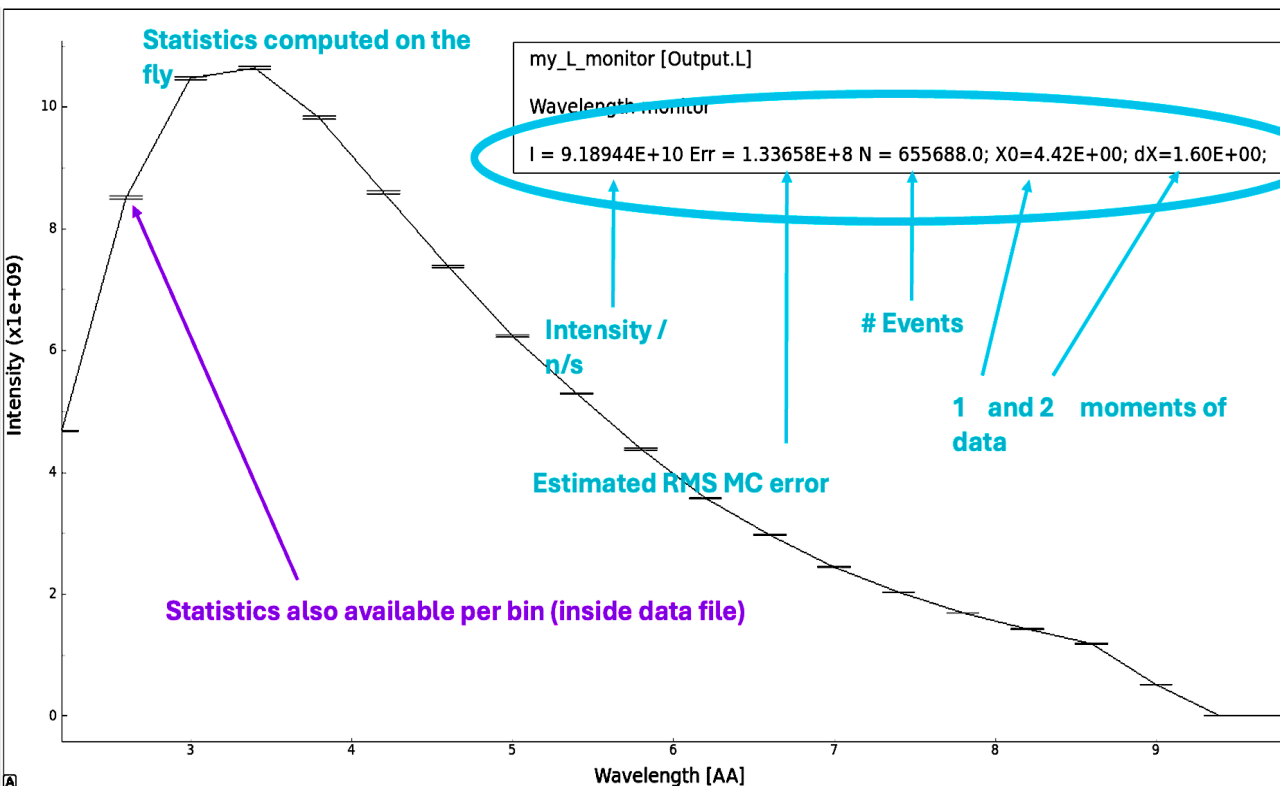
```
COMPONENT MyMon = Monitor_nD(xwidth=0.1, yheight=0.1, zdepth=0,  
    options="intensity per cm2 angle, limits=[-5 5], bins=10, with borders, file=mon1")
```

```
options = "banana, theta limits=[10,130], bins=120, y"
```

```
options = "user1, auto"    → with user1="age", username1="Age of the Captain [years]" — monitor just about  
anything.
```



# 1D and 2D example output (by default in Ascii)



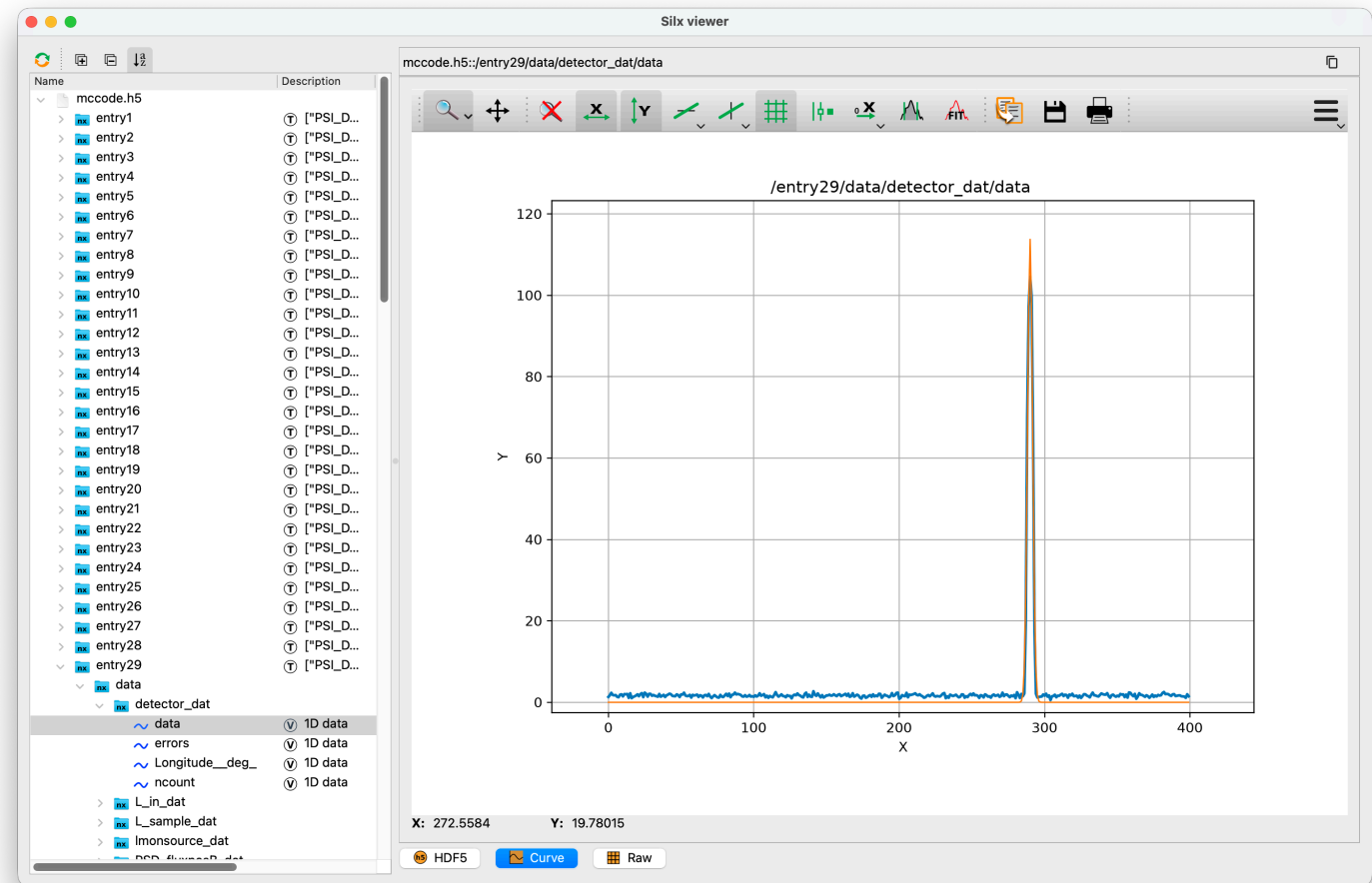
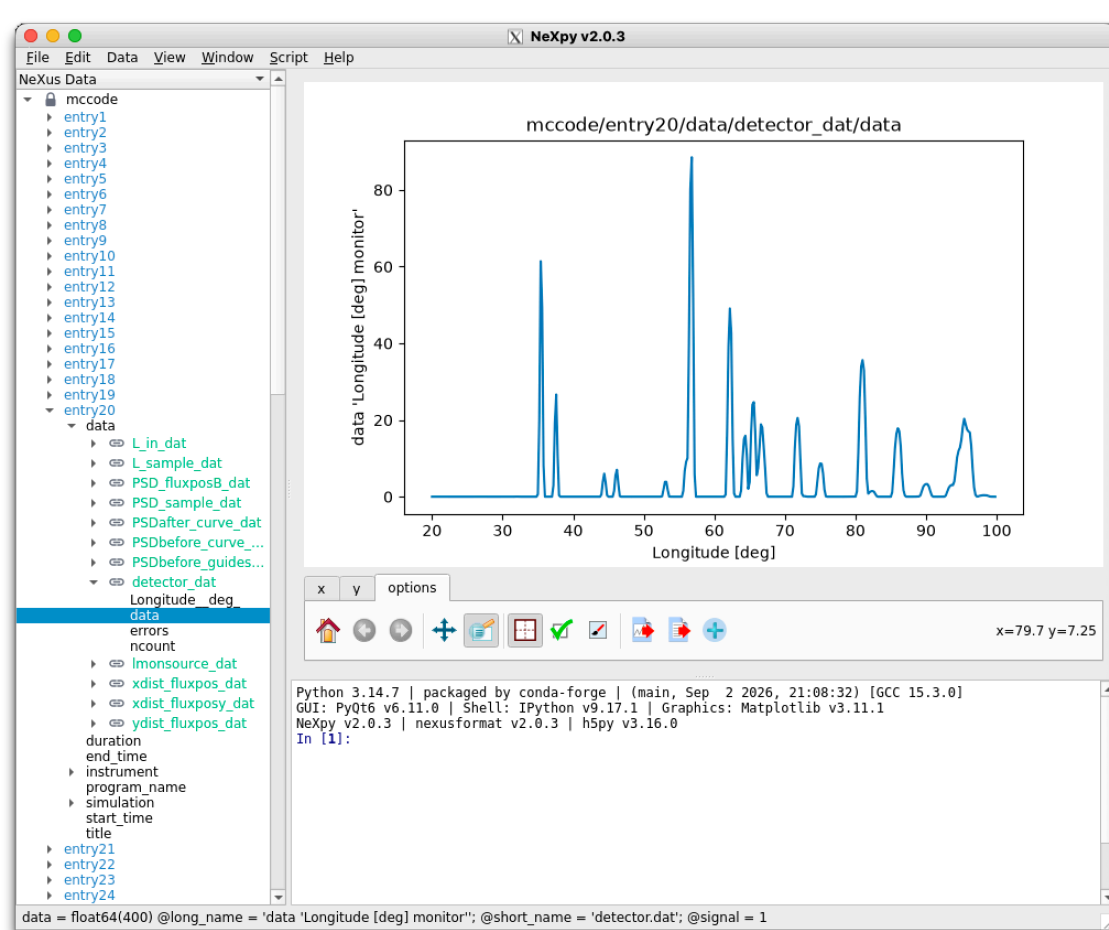
Plotting tools are [mcplot](#)(-pyqtgraph/-matplotlib/-html) for McStas and [mxplot](#)(-pyqtgraph/-matplotlib/-html) for McXtrace

There are further [mccoplot](#)/[mxcoplot](#) and [mcplotdiff](#)/[mxplotdiff](#) tools

When output is written in HDF5 / NeXus format we recommend the '[nexpy](#)' software

(always included when installing from conda-forge)

# Lightweight HDF5/NeXus plotting, fits etc.



NeXpy



# Sources differ; monitors mostly don't

## What's shared

- A source's role: set the ray state from chosen distributions
- The solid-angle / ncount / weight formalism
- Simple analytic sources: Source\_simple  $\leftrightarrow$  Source\_flat
- The monitor library, naming, and statistics engine
- Monitor\_nD and its options syntax
- MCPL for coupling simulations together
- mcdoc / mxdoc as the doc-access pattern

## What differs

- Source families: pulsed/reactor moderators vs. accelerator insertion devices
- Ray fields: velocity & spin vs. wave vector, phase & E-field
- Energy scale of the emitted spectrum: meV vs. keV
- A handful of probe-specific monitors (e.g. polarisation, neutron-only)
- Facility-specific source components (ISIS, SNS, ESS ... have no X-ray analogue)

# Where to go deeper

## McStas

`mcdoc / mcdoc -c`

[mcstas.org/download/components](https://mcstas.org/download/components)

[mcstas-users@mcstas.org](mailto:mcstas-users@mcstas.org)

## McXtrace

`mxdoc / mxdoc -c`

[mcxtrace.org/download/components](https://mcxtrace.org/download/components)

[mcxtrace-users@mcxtrace.org](mailto:mcxtrace-users@mcxtrace.org)

*Hands-on exercises on sources and monitors will be provided during the school session.*

---

Peter Willendrup · [pkwi@fysik.dtu.dk](mailto:pkwi@fysik.dtu.dk) · DTU Physics